

DMRO: 환경 변화에 적응하는 작업 오프로딩

DMRO: A Deep Meta Reinforcement Learning-based Task Offloading Framework for Edge-Cloud Computing

Guanjin Qu and Huaming Wu (2020)

Local·edge·cloud 배치와 Q-learning, meta-initialization의 두 학습 시간축

왜 실행 위치를 작업마다 결정해야 하는가

위치	장점	비용과 제약
Local	전송 없이 바로 실행	IoT 계산 능력과 에너지 제약
Edge	장치와 가깝고 대역폭이 큼	cloud보다 계산 자원이 작을 수 있음
Cloud	가장 큰 계산 자원	먼 통신 경로와 낮은 대역폭 가능성

가장 빠른 서버가 항상 가장 작은 전체 비용을 만드는 것은 아니다.

논문의 workflow는 순차 의존 작업으로 표현된다

각 task의 출력 data가 다음 task의 입력으로 이어진다



일반적인 임의 DAG 전체가 아니라, 논문 식 (1)의 순차 workflow가 모델 대상이다.

각 task는 정확히 한 위치를 선택한다

행동은 binary offload가 아니라 세 가지 discrete placement다

행동 a_i	배치 벡터	실행 위치	의미
0	$[1, 0, 0]$	Local	IoT device에서 실행
1	$[0, 1, 0]$	Edge	edge server로 offload
2	$[0, 0, 1]$	Cloud	cloud server로 offload

한 task를 여러 위치에 나누어 실행하는 partial partition 모델은 아니다.

선행 task 위치가 전송 지연을 결정한다

선행 위치	현재 위치	전송 지연
같은 위치	같은 위치	0
Local	Edge	$\text{data size} \div \text{IoT-edge bandwidth}$
Edge	Cloud	$\text{data size} \div \text{edge-cloud bandwidth}$
Local	Cloud	$\text{data size} \div \text{IoT-cloud bandwidth}$

현재 task의 최선은 선행 task가 어디에 있었는지에 따라 달라질 수 있다.

목적 함수는 지연과 에너지를 한 비용으로 묶는다

01	계산 지연	task work를 선택 위치의 계산 능력으로 나눈다
↓		
02	전송 지연	위치가 바뀌면 data size를 link bandwidth로 나눈다
↓		
03	에너지	task 크기와 위치별 energy 계수를 곱한다
↓		
04	Workflow 비용	전체 지연에 $\delta \times$ 전체 energy를 더한다
↓		
05	배치 선택	모든 task가 세 위치 중 하나를 고르도록 최소화한다

δ 는 energy와 delay 사이의 상대적 중요도를 조절한다.

비용 모델이 포함하는 것과 제외하는 것

항목	모델 처리	발표에서의 해석
계산 지연	위치별 computing capacity 반영	task work가 클수록 증가
전송 지연	인접 task 위치가 다르면 반영	link bandwidth가 작으면 증가
계산 에너지	위치별 energy 계수 반영	$\alpha \cdot \beta$ 로 서버 energy 가중
전송 에너지	단순화를 위해 제외	실제 장치 비용 전체는 아님
결정 지연	짧다고 가정하여 제외	실시간 latency 측정은 아님

최적이라는 표현은 이 논문이 정의한 비용과 가정 안에서만 성립한다.

교육용 계산: cloud가 가장 빠르는데 edge가 이길 수 있다

논문 결과가 아닌 단일 task 예시이며 energy는 제외한다

선택	계산 지연	추가 전송	합계
Local	$1000 \div 100 = 10.0 \text{ s}$	0 s	10.0 s
Edge	$1000 \div 500 = 2.0 \text{ s}$	0.1 s	2.1 s
Cloud	$1000 \div 1000 = 1.0 \text{ s}$	2.0 s	3.0 s

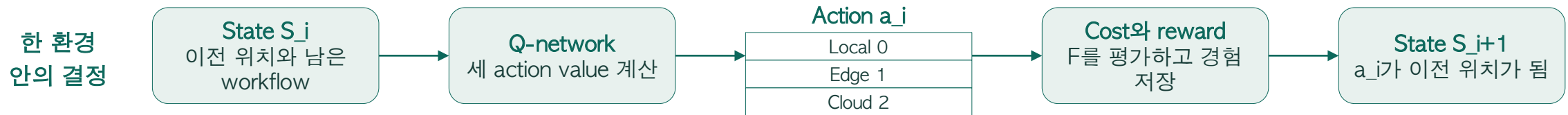
서버 계산 속도와 network 이동 비용을 함께 봐야 placement가 정해진다.

한 task의 결정을 RL transition으로 바꾼다

요소	논문의 정의	필요한 이유
State S_i	이전 action, incoming flow, 현재와 남은 task 정보	앞선 배치와 남은 일을 함께 본다
Action a_i	0 local, 1 edge, 2 cloud	현재 task의 위치를 고른다
Local cost F	계산 지연 + 이전 전송 지연 + $\delta \times \text{energy}$	현재 action을 비용으로 비교한다
Reward R	최적 action: $-\min F$; 그 외: $-\max F$	원문 고유의 두 수준 reward 규칙
Next state	현재 action이 다음 state의 이전 위치가 됨	의존 전송비용을 다음 결정에 전달한다

Meta-learning이 action을 고르는 것이 아니라 inner Q-network가 이 transition을 학습한다.

Inner loop: 이전 배치가 다음 결정으로 전달된다



Task를 순서대로 처리하므로 현재 action은 다음 state의 일부가 된다.

교육용 trace: 두 task의 배치가 이어지는 방식

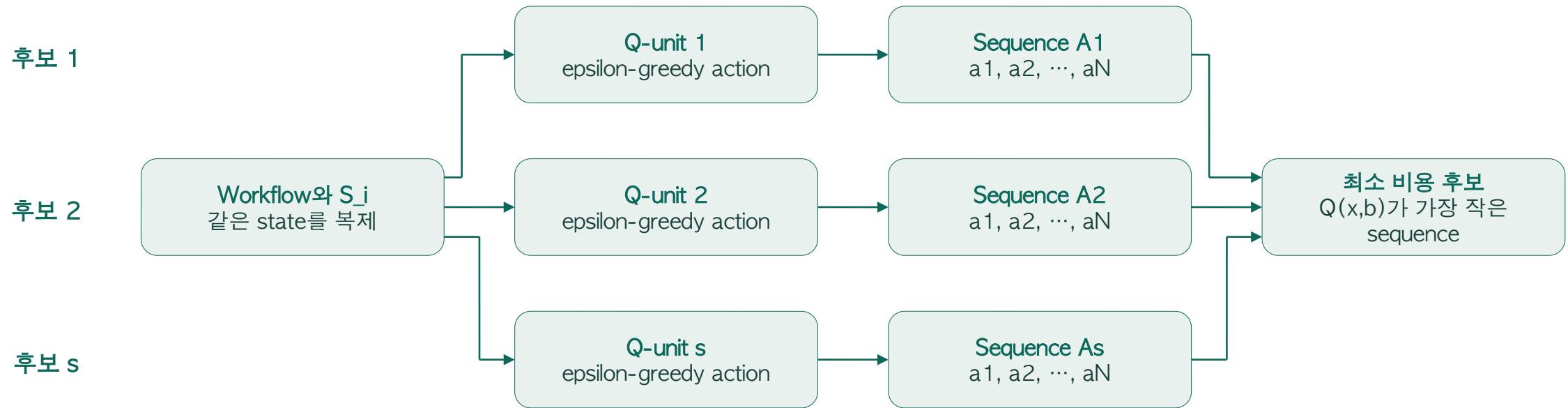
논문 결과가 아닌 계산 예시이며, 각 단계에서 energy는 0으로 둔다

단계	State의 이전 위치	비교한 총 지연	선택과 다음 state
Task 1	시작 위치 Local	Local 10.0, Edge 2.1, Cloud 3.0 s	Edge 선택 → 이전 위치 Edge
Task 2	이전 위치 Edge	Local 5.2, Edge 1.0, Cloud 0.9 s	Cloud 선택 → 이전 위치 Cloud

두 번째 비교에는 첫 번째 선택에서 생긴 위치 정보가 실제로 들어간다.

여러 Q-network가 서로 다른 전체 배치 후보를 만든다

s개의 parallel unit은 물리적으로 분리된 서버라는 뜻이 아니다



병렬성의 역할은 한 workflow에 대해 여러 placement sequence를 탐색하는 것이다.

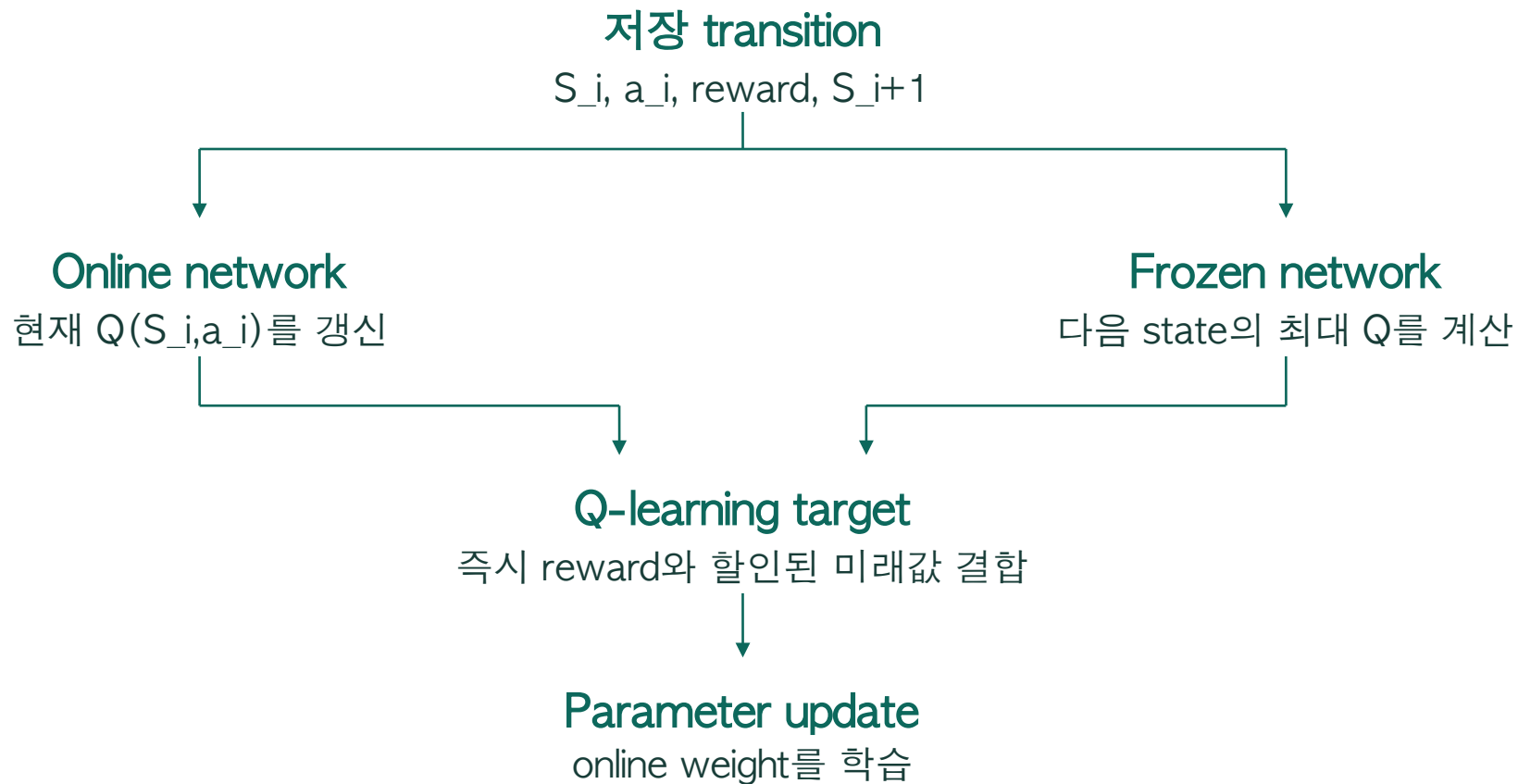
Algorithm 1: 후보 생성과 Q-learning을 함께 수행한다

원문 26개 줄을 학습 흐름 중심으로 압축한 구조다

01	Meta 초기화	s개 DNN을 같은 psi로 초기화하고 memory를 비운다
↓		
02	Task별 action	S_j를 복제하고 각 unit이 epsilon-greedy action을 낸다
↓		
03	후보 완성	각 unit의 action을 A_j에 누적해 전체 배치를 만든다
↓		
04	최종 선택	Workflow 비용이 가장 작은 A_j를 출력한다
↓		
05	Replay 학습	경험을 저장하고 표본화해 s개 network를 갱신한다

최종 후보 선택과 network 학습은 관련되지만 같은 연산 단계는 아니다.

Replay memory와 frozen target은 학습 신호를 분리한다



식 (18): 현재 추정과 bootstrap target을 섞는다

항	역할	원문 기호
현재 추정	Online network가 계산한 $Q(S_i, a_i)$	Q
즉시 신호	선택한 action에서 받은 reward	R
미래 추정	Frozen network의 next-state 최대 Q	$\max Q$
학습률	새 target을 반영하는 비율	θ
할인율	미래 reward의 비중	μ

이 업데이트는 한 환경의 action value를 학습하며, ψ 를 직접 만드는 식은 아니다.

논문의 DQN baseline과 DMRO의 차이

구성	DQN baseline	DMRO
후보 network	Parallel network 1개로 단순화	s개 parallel unit
새 환경 시작	일반 초기 parameter	학습된 meta-parameter ψ
Action	Local·edge·cloud 중 선택	동일한 세 action
Inner update	Q-learning	Replay와 frozen target을 쓰는 Q-learning
검증 범위	Fig. 6과 8의 simulator baseline	같은 simulator 조건의 제안법

DMRO의 차별점은 action 공간이 아니라 병렬 후보와 환경 간 초기화 학습이다.

새 환경마다 처음부터 학습하는 문제

- | | | |
|----|----------|-----------------------------------|
| 01 | 환경 변화 | 계산 능력과 link bandwidth 조건이 달라진다 |
| ↓ | | |
| 02 | 비용 관계 변화 | 같은 task라도 좋은 placement가 달라질 수 있다 |
| ↓ | | |
| 03 | 기존 초기화 | 한 환경에 맞춘 weight가 새 환경에 맞지 않을 수 있다 |
| ↓ | | |
| 04 | Meta 목표 | 여러 환경에서 빠르게 학습할 시작점 psi를 만든다 |

DMRO에서 meta-learning의 출력은 placement가 아니라 inner network의 초기값이다.

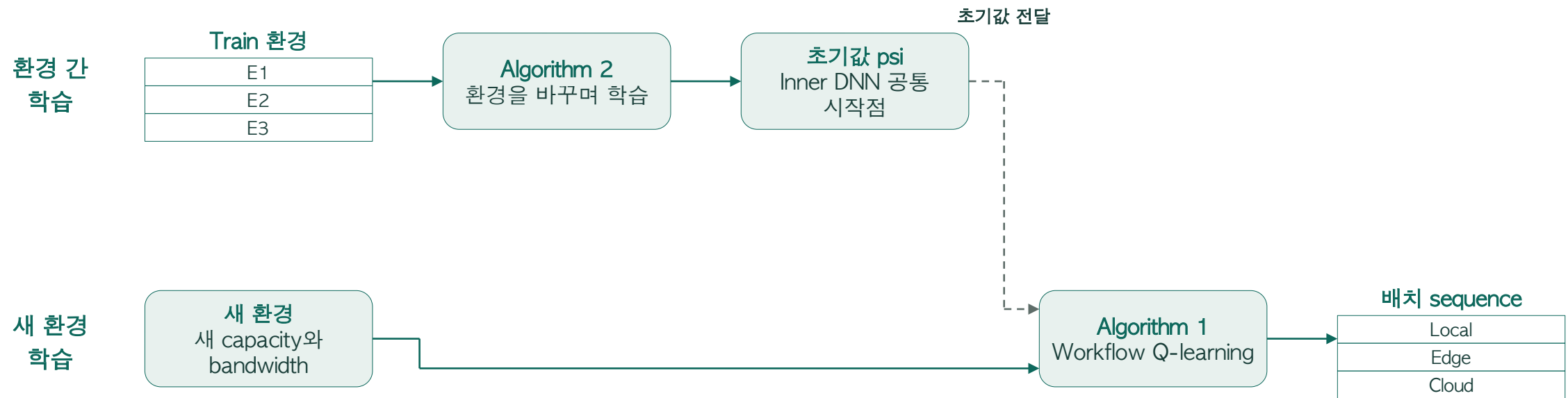
Algorithm 2: 여러 환경을 바꾸며 초기값 ψ 를 학습한다

Meta algorithm의 출력은 최종 DNN parameter다

01	일반 초기화	DNN을 θ_0 로 시작하고 memory를 비운다
↓		
02	환경 표본화	한 환경을 무작위로 골라 state와 action을 만든다
↓		
03	경험 저장	환경에서 얻은 transition을 memory에 넣는다
↓		
04	환경 재표본화	환경을 다시 고르고 random replay로 weight를 갱신한다
↓		
05	Meta 출력	훈련된 DNN parameter를 ψ 로 내보낸다

Outer loop는 특정 환경의 최적점보다 여러 환경에 쓸 시작점을 목표로 한다.

두 시간축: 환경 간 초기화와 환경 안의 적응



Meta-learning은 action selector가 아니라 inner learner의 시작점을 만든다.

DMRO의 meta algorithm과 표준 MAML을 구분한다

항목	MAML 원 논문	DMRO 원문
공통 목표	적은 update로 적응할 초기 parameter	새 MEC 환경에 쓸 초기 parameter ψ
Task 또는 환경	Task distribution에서 표본화	서로 다른 MEC environment를 표본화
Inner update	Task별 gradient adaptation	Q-learning 경험으로 DNN 갱신
Outer update	Adaptation 뒤 meta-gradient 명시	별도 meta-gradient 식을 제시하지 않음
안전한 명칭	Model-Agnostic Meta-Learning	MAML에서 영감을 받은 초기화 학습

빠른 적응이라는 목표는 공유하지만 Algorithm 2를 그대로 MAML이라 부를 수는 없다.

실험은 simulator의 한 edge·한 cloud 환경이다

항목	설정	해석 경계
사용자와 workflow	IoT user 4명, 각 5개 workflow	실장치 trace가 아님
Task data	첫 task 50–100 MB, 이후 10–50 MB	무작위 생성 workload
Task work	10^3 – 10^5 MHz로 무작위 표기	원문 단위를 그대로 인용
DNN	입력층, hidden layer 2개, 출력층	세부 layer 폭은 제시되지 않음
Energy weight	$\alpha = \beta = 1$	서버 energy도 동일 weight로 포함

결과는 실제 edge hardware benchmark가 아니라 설정된 simulator 비교다.

Table 1의 기준 환경

자원	Local	Edge	Cloud 또는 link
Computing capacity	30 MHz	70 MHz	150 MHz
Bandwidth	IoT-edge 800 MB/s	edge-cloud 200 MB/s	IoT-cloud 10 MB/s
Energy	0.3 J/MB	0.15 J/MB	0.1 J/MB

이 설정에서는 cloud의 계산은 빠르지만 IoT-cloud link는 가장 느리다.

수렴 실험이 알려 주는 training 설정

그림	비교	원문 관찰	후속 설정
Fig. 4	여러 learning rate	0.01이 표시된 조건 중 가장 좋은 수렴	이후 0.01 사용
Fig. 5	Batch 128, 256, 512, 1024	Batch가 클수록 curve 변동이 작음	단일 최적 batch 단정 없음
Fig. 5	Target copy 주기	200 step마다 작은 fluctuation	200 step마다 parameter 복사

수렴 곡선은 설정 선택 근거이지 실제 의사결정 시간 측정이 아니다.

Fig. 6은 다섯 배치 방식의 목적 함수 값을 비교한다

방식	배치 규칙	비교에서의 역할
Local-only	모든 task를 device에서 실행	전송 없는 고정 baseline
Edge-only	모든 task를 edge에 배치	Edge full-offload baseline
Cloud-only	모든 task를 cloud에 배치	Cloud full-offload baseline
DQN	한 Q-network로 동적 배치	DMRO의 단순화 baseline
DMRO	병렬 Q-network와 meta 초기값	제안 방식

표시된 weight-ratio 조건에서 DMRO가 다섯 방식 중 가장 낮은 총비용을 보인다.

Fast-learning 실험은 train과 test 환경을 다르게 둔다

Parameter	Train 범위	Test 값
Computing capacity	Local 15–25, edge 50–60, cloud 160–170 MHz	30, 70, 150 MHz
IoT-edge bandwidth	600–700 MB/s	800 MB/s
Edge-cloud bandwidth	100–150 MB/s	200 MB/s
IoT-cloud bandwidth	20–30 MB/s	10 MB/s
Energy coefficient	0.3, 0.15, 0.1 J/MB	동일

Meta parameter는 학습 범위와 다른 고정 test environment에서 평가된다.

Fig. 7과 8: learned initialization의 조건부 이점

그림	비교 대상	보고된 관찰	말할 수 없는 것
Fig. 7	Meta parameter vs 일반 초기화	초기 수렴이 좋고 더 빠르게 수렴	모든 환경의 보장
Fig. 8	같은 두 초기화	몇 차례 학습 뒤 더 낮은 offloading cost	정확한 실시간 speedup
공통 조건	Table 2의 train/test shift	Simulator에서 portability를 지지	Hardware deployment 검증

근거는 특정 환경 이동에서의 curve 비교이며, adaptation theorem은 아니다.

논문의 결과를 안전하게 읽는 경계

주장

근거가 있는 범위

확장하면 안 되는 해석

낮은 비용

Fig. 6의 simulator와 다섯 baseline

전 환경의 global optimum

빠른 적응

Table 2 shift에서 Fig. 7-8 비교

재학습이 전혀 필요 없음

병렬 DNN

s개 후보 network의 algorithm 구조

서로 다른 machine의 분산 실행

실시간 decision

결정 지연을 짧다고 가정

측정된 end-to-end latency

확장성

저자가 multi-server 확장을 예상

구현과 실험으로 검증됨

이 논문은 시뮬레이션 근거를 제공하지만 최적성·실장치·확장성 보장은 제공하지 않는다.

DMRO를 한 흐름으로 다시 읽기

- | | | |
|----|------------------|---|
| 01 | 의존 workflow | 각 task와 중간 data flow를 순서대로 입력한다 |
| ↓ | | |
| 02 | 세 위치 비용 | 계산·전송 지연과 weighted energy를 평가한다 |
| ↓ | | |
| 03 | Inner Q-learning | 병렬 network가 placement sequence 후보를 만든다 |
| ↓ | | |
| 04 | Outer 초기화 | 여러 환경에서 새 learner의 시작점 psi를 학습한다 |
| ↓ | | |
| 05 | 조건부 평가 | 정해진 simulator와 environment shift에서 비교한다 |

DMRO의 핵심은 세 위치의 순차 결정과 환경 간 초기화 학습을 분리해 결합한 것이다.